

throughput and enhance scalability in applications with a large percentage of I/O tasks, as well as for real-time applications such as online gaming and communication. It offers a set of packages aimed at making the design of Web servers much easier, with a set of modules that handle file systems, I/O, data streams, cryptography and other core functionality. Thus, Node.js is both a runtime environment and a library, which runs on Google's V8 VM—the same one that Chrome uses.

Node.js has found widespread application in organisations including Go Daddy, IBM, Walmart, Tauten, SAP, Yahoo and LinkedIn, among others.

Getting hands-on with Node.js

Let's try our hand at building a Node.js server.

Installation: The popularity of Node.js has resulted in multiple versions often being installed on the same system for compatibility or separate projects. This calls for the need to manage these versions, which is where the node version manager (or nvm) comes into the picture. Note that you must have the following prerequisites installed.

For OS X:

```
$ xcode-select install
```

For Linux:

```
$ sudo apt-get update
$ sudo apt-get install build-essential
```

Check out the Windows installer at github.com/creationix/nvm, and install it on your Linux/OS X system by using curl (or wget), as follows:

```
$ curl https://raw.githubusercontent.com/creationix/nvm/v0.33.8/install.sh | bash
```

(The version at the time of writing is 0.33.8, but this may change later.)

2. Now that we have nvm



Figure 1: The pillars of Web design [Credits: Bradley Nice]



Figure 2: Java vs JavaScript [Credits: gm-dude]



Figure 3: Node.js [Credits: donttouchthespikes.com]

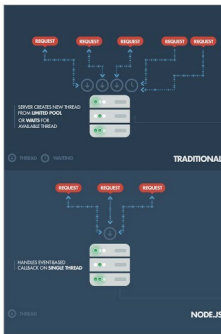


Figure 4: Node.js server vs conventional Web server [Credits: toptal.com]

installed, we can go ahead and install Node.js, as follows:

```
$ nvm install
```

The above command pulls the latest version of Node.js for installation on the system. You can also append the version you require to the above command.

You should now have a working install of Node.js:

```
$ which node
```

As mentioned earlier, Node.js has a number of modules for providing a diverse set of functionalities and making it easier to build an application. Well, there's a package manager to handle installation, storage, and the removal of these packages from your system, called the node package manager (npm). You can run the following command to verify this:

```
$ which npm
```

Building a Node.js server

We start with creating a directory for the project and making it the working directory, as follows:

```
$ mkdir myapp
$ cd myapp
```

When you try to build your first server, the 'http' module is the first one you will interact with, in order to bind to a particular port as shown below. Let us create a file called *index.js* and add the following code to it:

```
$ touch index.js
```

As you can see, with the above code, we are starting to build a server using the 'http' module, which we direct to listen on Port 4000 for incoming requests. If there is an error, we print it to the console for debugging. That's it. Now, simply run the following command: